Travaux Pratiques Python

BTS CIEL — 1re année

Lycée Denis Diderot — Année 2025–2026

NOM : _____

Prénom : _____

Groupe : _____

TP	Thème	Note /10
1	Variables et types de données	
2	Structures conditionnelles	
3	Boucles `for` et `while`	
4	Fonctions	
5	Listes	
6	Chaînes de caractères	
7	Dictionnaires	
8	Fichiers et gestion d'erreurs	
9	Modules `numpy` et `matplotlib`	
10	Projet : calculatrice électronique	
	Total	**/100**

Chaque TP est noté sur 10 points. Le document de réponse est à rendre en fin de séance ou à déposer sur le réseau pédagogique.

TP 1 — Variables et types de données /10 pts

Contexte : calcul électronique avec la loi d'Ohm $U = R \times I$ $P = U \times I$

Objectif

Déclarer des variables, utiliser les types `int`, `float`, `str`, effectuer des calculs et afficher des résultats formatés.

Exemple : déclaration et affichage

```python
R = 470            # résistance en ohms (int)
I = 0.02           # courant en ampères (float)
nom = "Circuit"    # chaîne de caractères (str)
U = R * I
print(f"Tension U = {U} V")
print(type(R))     # affiche <class 'int'>
```

Question 1 (2 pts) – Ouvrir un nouveau fichier Python nommé `tp1.py`. Déclarer les variables `R = 470` et `I = 0.02`, puis calculer et afficher la tension `U`. Vérifier que le résultat est $U = 9{,}4$ V.

Question 2 (2 pts) – Utiliser la fonction `type()` pour afficher le type de `R`, `I` et d'une variable `nom = "Résistance"`. Indiquer ci-dessous les trois types obtenus.

Question 3 (2 pts) – Demander à l'utilisateur de saisir la valeur de `R` et de `I` avec `input()`. Convertir les saisies en `float`, recalculer `U` et l'afficher.

Question 4 (2 pts) – Calculer la puissance dissipée $P = U \times I$. Afficher le résultat arrondi à 4 décimales avec `round()` et avec un f-string indiquant l'unité (watts).

Question 5 (2 pts) – Compléter le programme pour afficher un résumé formaté avec des f-strings sur plusieurs lignes : `R = ... Ω | I = ... A | U = ... V | P = ... W`

Appeler le professeur pour vérification avant de passer à la suite.

TP 2 — Structures conditionnelles /10 pts

Contexte : gestion de seuils dans un circuit électronique

Objectif

Utiliser `if`, `elif`, `else` et les opérateurs logiques `and`, `or`, `not` pour prendre des décisions dans un programme.

Aide / Rappel

```python
if condition:
    # bloc exécuté si vrai
elif autre_condition:
    # sinon si ...
else:
    # sinon
```

Opérateurs de comparaison : ==, !=, <, >, <=, >=

Question 1 (2 pts) – Une LED rouge est passante si la tension à ses bornes dépasse 2,0 V. Écrire un programme qui demande la tension `U` et affiche `"LED allumée"` ou `"LED éteinte"` selon le cas.

Question 2 (2 pts) – Un régulateur de tension classifie la tension de sortie en 3 zones : $U < 4{,}5$ V : **tension faible**, $4{,}5 \leq U \leq 5{,}5$ V : **tension nominale**, $U > 5{,}5$ V : **tension élevée**. Écrire le programme avec `elif`.

Question 3 (2 pts) – Un système de protection se déclenche si la tension dépasse 12 V **ET** si le courant dépasse 2 A. Écrire le programme utilisant `and`. Tester avec $(U = 13, I = 1)$, $(U = 13, I = 3)$, $(U = 10, I = 3)$.

Question 4 (2 pts) – Écrire un programme qui affiche la table de vérité de la porte logique AND pour deux entrées `A` et `B` valant 0 ou 1. Utiliser deux boucles `for` imbriquées et une condition.

Question 5 (2 pts) – Un thermostat coupe le chauffage si $T > 22°C$ et l'allume si $T < 18°C$. Entre les deux, il maintient l'état précédent (afficher `"Maintien"`). Demander T à l'utilisateur et afficher l'action.

Appeler le professeur pour vérification.

TP 3 — Boucles `for` et `while` /10 pts

Contexte : répétitions, comptages, simulations de mesures

Objectif

Maîtriser les boucles `for` (nombre d'itérations connu) et `while` (condition de sortie), utiliser `break`, `continue` et `range()`.

Exemple : exemples de boucles

```python
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 10, 2):   # 1, 3, 5, 7, 9
    print(i)

compteur = 0
while compteur < 3:
    print(compteur)
    compteur += 1
```

Question 1 (2 pts) – Afficher les 12 premières valeurs de la série E12 de résistances en multipliant une valeur de départ par $\sqrt[12]{10} \approx 1{,}2115$. Partir de $R_0 = 10\ \Omega$ et afficher chaque valeur arrondie à l'entier.

Question 2 (2 pts) – Écrire une boucle `while` qui commence avec une tension $U = 100$ V et la divise par 2 à chaque itération jusqu'à ce qu'elle soit inférieure à 1 V. Afficher le nombre d'itérations nécessaires.

Question 3 (2 pts) – Écrire un programme qui calcule la somme des n premiers entiers $1 + 2 + \cdots + n$ avec une boucle `for`. Demander n à l'utilisateur. Vérifier avec la formule $\dfrac{n(n+1)}{2}$.

Question 4 (2 pts) – Programme de devinette : l'ordinateur choisit un nombre entre 1 et 100 avec `random.randint(1, 100)`. L'utilisateur propose des valeurs jusqu'à trouver. Afficher le nombre de tentatives.

Question 5 (2 pts) – Afficher la table de multiplication des résistances de 1 à 5 (comme pour $R_{\text{série}} = R_1 + R_2$, toutes combinaisons). Utiliser deux boucles `for` imbriquées et aligner les colonnes avec `:>4d`.

Appeler le professeur pour vérification.

TP 4 — Fonctions /10 pts

Contexte : bibliothèque de calculs électroniques réutilisables

Objectif

Définir des fonctions avec `def`, passer des paramètres, utiliser `return`, les valeurs par défaut et les fonctions renvoyant plusieurs résultats.

Exemple : définition et appel d'une fonction

```python
def loi_ohm(R, I):
    """Retourne la tension U = R * I"""
    U = R * I
    return U

# Appel
tension = loi_ohm(470, 0.02)
print(f"U = {tension} V")
```

Question 1 (2 pts) – Écrire une fonction `loi_ohm(R, I)` qui retourne la tension $U = R \times I$. La tester avec $R = 1000\ \Omega$ et $I = 0{,}015$ A.

Question 2 (2 pts) – Écrire une fonction `bilan_puissance(R, I)` qui retourne un **tuple** (U, P) où $U = R \times I$ et $P = R \times I^2$. L'appeler et décompacter le résultat dans deux variables.

Question 3 (2 pts) – Écrire une fonction `resistance_serie(*args)` qui accepte un nombre quelconque de résistances et retourne leur somme. Tester avec 3, puis 5 résistances.

Question 4 (2 pts) – Écrire une fonction `resistance_parallele(R1, R2)` qui calcule $R_p = \dfrac{R_1 \times R_2}{R_1 + R_2}$. Ajouter une valeur par défaut `R2=100`. Tester avec $R_1 = 200\ \Omega$ sans préciser R_2.

Question 5 (2 pts) – Créer une fonction `convertir_tension(U, unite="V")` qui convertit une tension selon l'unité demandée (`"mV"`, `"V"` ou `"kV"`) et retourne la valeur convertie. Tester les 3 cas.

Appeler le professeur pour vérification.

TP 5 — Listes /10 pts

Contexte : traitement d'une série de mesures de tension

Objectif

Créer, indexer, modifier des listes ; utiliser les méthodes `append`, `remove`, `sort` ; parcourir une liste avec `for` ; utiliser `min`, `max`, `sum`, `len`.

Exemple : manipulation de listes

```python
mesures = [4.98, 5.02, 4.95, 5.10, 4.88]
print(mesures[0])       # premier élément : 4.98
print(mesures[-1])      # dernier : 4.88
mesures.append(5.05)    # ajout en fin
print(len(mesures))     # longueur : 6
```

Question 1 (2 pts) – Créer une liste `mesures` contenant les 8 valeurs suivantes (en volts) : `[4.98, 5.02, 4.95, 5.10, 4.88, 5.01, 4.97, 5.08]`. Afficher le premier élément, le dernier et l'élément d'indice 4.

Question 2 (2 pts) – Ajouter la valeur `5.15` avec `append()`. Supprimer la valeur `4.88` avec `remove()`. Afficher la liste mise à jour et sa longueur.

Question 3 (2 pts) – Calculer et afficher la **moyenne**, la valeur **minimale** et la valeur **maximale** de la liste `mesures`. Utiliser `sum()`, `min()`, `max()` et `len()`.

Question 4 (2 pts) – Créer une nouvelle liste `hors_limites` contenant uniquement les mesures qui sont **en dehors** de l'intervalle [4,90 V ; 5,10 V]. Utiliser une boucle `for` et `append()`.

Question 5 (2 pts) – Trier la liste `mesures` avec `sort()` et afficher la liste triée. Extraire ensuite le **premier tiers** de la liste triée avec le slicing (`liste[debut:fin]`).

Appeler le professeur pour vérification.

TP 6 — Chaînes de caractères /10 pts

Contexte : analyse de trames de communication série (protocole ASCII)

Objectif

Manipuler les chaînes de caractères : concaténation, indexation, méthodes `upper`/`lower`, `split`, `strip`, `startswith`, `find`, formatage avec f-strings.

Exemple : méthodes principales

```python
trame = "START:12.5:STOP"
print(trame.upper())            # START:12.5:STOP
print(trame.split(":"))         # ['START', '12.5', 'STOP']
print(trame.startswith("ST"))   # True
print(len(trame))               # 15
print(trame[6:9])               # 12.
```

Question 1 (2 pts) – Déclarer `trame = "START:12.5:STOP"`. Afficher sa longueur, le 1er caractère, le dernier caractère et les caractères d'indices 6 à 8 (slicing).

Question 2 (2 pts) – Décomposer la trame avec `split(":")`. Stocker le résultat dans une liste `parties`. Afficher séparément le marqueur de début, la valeur et le marqueur de fin.

Question 3 (2 pts) – Écrire une fonction `valider_trame(t)` qui retourne `True` si la trame commence par `"START"` **et** se termine par `"STOP"`, `False` sinon. Tester avec 3 trames différentes.

Question 4 (2 pts) – Construire un message de log formaté avec f-string : `"[INFO] Capteur 3 - Valeur : 12.50 V - Statut : OK"` à partir de variables `num=3`, `val=12.5`, `statut="OK"`.

Question 5 (2 pts) – Écrire une fonction `compter_char(chaine, car)` qui compte le nombre d'occurrences du caractère `car` dans `chaine` sans utiliser `.count()`. Tester en comptant les `":"` dans la trame.

Appeler le professeur pour vérification.

TP 7 — Dictionnaires /10 pts

Contexte : catalogue de composants électroniques

Objectif

Créer, lire, modifier des dictionnaires ; utiliser `keys()`, `values()`, `items()` ; travailler avec des dictionnaires imbriqués ; rechercher des valeurs.

Exemple : dictionnaire simple

```python
composant = {
    "référence": "R470",
    "valeur": 470,
    "unité": "Ohm",
    "tolérance": 5
}
print(composant["valeur"])    # 470
composant["tolérance"] = 1    # modification
```

Question 1 (2 pts) – Créer un dictionnaire `resistances` avec 4 résistances. Chaque clé est la référence (`"R1"`, `"R2"`, ...) et la valeur est la résistance en ohms. Afficher la valeur de `"R3"` et le nombre total d'entrées.

Question 2 (2 pts) – Ajouter une résistance `"R5": 2200` avec l'opérateur `[]=`. Supprimer `"R2"` avec `del`. Vérifier si `"R2"` est encore dans le dictionnaire avec `in`.

Question 3 (2 pts) – Parcourir le dictionnaire `resistances` avec `.items()` et afficher, pour chaque résistance, la référence et la valeur au format : `Référence : R1 | Valeur : 470 Ω`

Question 4 (2 pts) – Créer un dictionnaire `catalogue` imbriqué où chaque composant possède les attributs `valeur`, `tolérance` et `stock`. Afficher le stock du 2e composant.

Question 5 (2 pts) – Écrire une fonction `chercher_stock(catalogue, seuil)` qui retourne la liste des références dont le stock est **inférieur** au seuil. Tester avec `seuil=10`.

Appeler le professeur pour vérification.

TP 8 — Fichiers et gestion d'erreurs /10 pts

Contexte : datalogger – enregistrement de mesures dans un fichier

Objectif

Ouvrir, lire, écrire des fichiers texte avec `open()` et le gestionnaire `with` ; utiliser le module `csv` ; gérer les erreurs avec `try / except`.

Exemple : lecture et écriture de fichier

```python
# Écriture
with open("mesures.txt", "w", encoding="utf-8") as f:
    f.write("5.02\n")
    f.write("4.98\n")

# Lecture
with open("mesures.txt", "r", encoding="utf-8") as f:
    for ligne in f:
        print(ligne.strip())
```

Question 1 (2 pts) – Écrire un programme qui crée un fichier `log.txt` et y écrit 5 lignes de mesures simulées (utiliser `random.uniform(4.9, 5.1)` pour générer des tensions aléatoires entre 4,9 V et 5,1 V).

Question 2 (2 pts) – Écrire un programme qui lit `log.txt` ligne par ligne, convertit chaque ligne en `float` et calcule la moyenne des valeurs lues. Afficher le résultat.

Question 3 (2 pts) – Utiliser le module `csv` pour créer un fichier `mesures.csv` avec les colonnes `index, temps_s, tension_V`. Générer 10 lignes avec `temps = index * 0.1` et une tension aléatoire.

Question 4 (2 pts) – Écrire un programme qui lit `mesures.csv` avec `csv.DictReader` et affiche uniquement les lignes où la tension est hors de l'intervalle [4,90 V; 5,10 V].

Question 5 (2 pts) – Entourer la lecture du fichier d'un bloc `try / except FileNotFoundError` pour afficher un message d'erreur clair si le fichier n'existe pas. Tester en donnant un nom de fichier inexistant.

Appeler le professeur pour vérification.

TP 9 — Modules `numpy` et `matplotlib` /10 pts

Contexte : visualisation d'un signal sinusoïdal – analyse fréquentielle

Objectif

Utiliser numpy pour les calculs vectoriels sur des tableaux et `matplotlib.pyplot` pour tracer des courbes. Générer un signal $u(t) = A\sin(2\pi f t)$ et le visualiser.

Exemple : signal sinusoïdal de base

```python
import numpy as np
import matplotlib.pyplot as plt

f = 50            # fréquence en Hz
A = 5             # amplitude en V
t = np.linspace(0, 0.04, 1000)    # 0 à 40 ms
u = A * np.sin(2 * np.pi * f * t)

plt.plot(t, u)
plt.show()
```

Question 1 (2 pts) – Créer un tableau de 1000 points de temps `t` sur la plage $[0\ s;\ 0{,}04\ s]$ avec `np.linspace`. Calculer $u(t) = 5\sin(2\pi \cdot 50 \cdot t)$ et afficher les 5 premières valeurs du tableau.

Question 2 (2 pts) – Tracer la courbe $u(t)$ avec `plt.plot()`. Ajouter un titre `"Signal sinusoïdal 50 Hz"`, des labels sur les axes (`"Temps (s)"` et `"Tension (V)"`), une grille avec `plt.grid()`.

Question 3 (2 pts) – Sur le même graphique, superposer le signal fondamental $u_1(t) = 5\sin(2\pi \cdot 50 \cdot t)$ et son harmonique 3 $u_3(t) = 1{,}5\sin(2\pi \cdot 150 \cdot t)$. Utiliser des couleurs différentes.

Question 4 (2 pts) – Calculer et tracer le signal composite $u(t) = u_1(t) + u_3(t)$ en ajoutant les deux tableaux numpy. Ajouter une légende avec `plt.legend()` pour identifier les 3 courbes.

Question 5 (2 pts) – Calculer les valeurs suivantes à partir du tableau u : valeur maximale (`np.max`), valeur efficace $U_{\text{eff}} = \sqrt{\text{moyenne}(u^2)}$ avec `np.sqrt` et `np.mean`. Afficher ces résultats. Vérifier que $U_{\text{eff}} \approx A/\sqrt{2}$.

Appeler le professeur pour vérification et validation du graphique.

TP 10 — Projet : calculatrice électronique interactive /10 pts

Bilan des compétences – application complète

Objectif

Concevoir une application complète en Python en mobilisant toutes les compétences acquises : fonctions, boucles, conditions, fichiers, formatage. Le programme doit être structuré, lisible et fonctionnel.

Aide / Rappel

Le projet est évalué sur la **qualité du code** (lisibilité, nommage, commentaires) autant que sur les fonctionnalités. Penser à tester chaque fonction individuellement avant de les assembler.

Question 1 (2 pts) – Écrire une fonction `menu()` qui affiche le menu principal suivant et retourne le choix de l'utilisateur (entier 1 à 4, ou 0 pour quitter) :

```
1 - Loi d'Ohm   2 - Diviseur de tension   3 - Résistances en parallèle   4 -
                    Résistance de LED   0 - Quitter
```

Intégrer la gestion du cas où la saisie n'est pas un entier (`try/except ValueError`).

Question 2 (2 pts) – Écrire la fonction `module_loi_ohm()` : demande deux des trois grandeurs (U, R, I) parmi celles que l'utilisateur connaît, calcule la troisième et affiche le résultat avec son unité.

Question 3 (2 pts) – Écrire la fonction `module_diviseur()` qui calcule la tension de sortie d'un pont diviseur : $U_s = U_e \times \dfrac{R_2}{R_1 + R_2}$. Demander U_e, R_1 et R_2 à l'utilisateur, calculer et afficher U_s.

Question 4 (2 pts) – Écrire la fonction `module_led()` qui calcule la résistance de limitation d'une LED : $R = \dfrac{U_{\text{alim}} - U_{\text{LED}}}{I_{\text{LED}}}$. Proposer des valeurs par défaut $U_{\text{LED}} = 2{,}0$ V et $I_{\text{LED}} = 0{,}02$ A.

Question 5 (2 pts) – Assembler toutes les fonctions dans une boucle principale `while True` qui affiche le menu, appelle la bonne fonction selon le choix, et enregistre chaque calcul effectué dans un fichier `historique.txt` avec la date et l'heure (`datetime.now()`).

Démonstration finale devant le professeur – présenter l'application en fonctionnement.

Critères de notation du TP 10 : fonctionnalité (6 pts) + qualité du code et commentaires (2 pts) + fichier historique opérationnel (2 pts).